1 Authentication

1.1 System overview

Native Rust implementation of the local Java module `authentication/`. Each named executable checks its configured role and owns an independent identity, listen port and state file. Shared Rust modules implement the runtime; no Java, Python or Go process implements the service.

1.2 Produced and consumed services

Produced HTTPS definitions: `echo`, `generalManagement`, `identity`, `identityManagement`. Service versions are 1.0.0; the target system version is 5.2.2. Authorities consumed as applicable are Authentication, ServiceRegistry and ConsumerAuthorization. Explicit pinned HTTPS origins and valid cloud identities must be provisioned.

1.3 Architecture and persistent state

The listener uses Tokio, Hyper/Axum and rustls with mandatory client certificates. Verified peer identity is supplied through a private request extension, never a declared-identity header. Business modules share locked JSON state with clone/validate/fsync/rename transactions. Credentials and keys are separate owner-only files. Cross-system operations and callbacks are not distributed transactions.

1.4 Security

Local cloud CN/profile, certificate chain and validity are verified. Identity tokens are checked against Authentication and bound to the client certificate. Management requires a verified sysop or explicit broad whitelist. Outbound peer chain/SAN and expected system CN/profile are checked before credentials are sent. Optional PEM CRLs are enforced at startup-configured trust boundaries and require process restart to reload. No plaintext HTTP listener, TLS bypass or declared SYSTEM identity mode is provided.

1.5 Implementation status and delimitations

This is a development profile, not a certified interchangeable Java replacement. HTTP route presence is not semantic equivalence. Java database migration, MQTT application APIs and production qualification remain outstanding. PASSWORD identities use bcrypt and single active opaque sessions. Java SQL/session import, complete query/time normalization and alternative framework transports require further work. No stored plaintext password or raw identity token is used.

2 Implementation structure

See `src/security.rs`, `src/store.rs`, `src/web.rs` and the role-specific module. Transactions clone state and publish only after durable rename. State locking prevents concurrent processes from sharing the same path. A directory-sync failure after rename is an uncertain durable commit, not an automatic rollback. HTTPS push callbacks are at-least-once across crash recovery; downstream effects can require reconciliation.

2.1 HTTPS interface operations

HTTPS uses JSON unless an operation specifies text. Request bodies are limited to 1 MiB; upstream responses to 8 MiB. Identity-token format: `Bearer IDENTITY-TOKEN//token`. Certificate-only identity operations still require verified mutual TLS and apply credential/certificate binding.

Method	Path	Access
GET	/authentication/general/mgmt/get-config	management
POST	/authentication/general/mgmt/logs	management
GET	/authentication/monitor/echo	certificate
POST	/authentication/identity/login	certificate
POST	/authentication/identity/logout	certificate
POST	/authentication/identity/change	certificate
GET	/authentication/identity/verify/{token}	identity
POST	/authentication/mgmt/identities	management
PUT	/authentication/mgmt/identities	management
DELETE	/authentication/mgmt/identities	management
POST	/authentication/mgmt/identities/query	management
DELETE	/authentication/mgmt/sessions	management
POST	/authentication/mgmt/sessions	management

Errors contain errorMessage, errorCode, exceptionType, origin and timestamp. Exact validation/error parity and all Java security-policy settings remain release gates. Unknown routes return 404; unavailable authorities fail closed.